

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Paralelní Programování v Go



2026

Vedoucí práce:
Mgr. Jan Laštovička, Ph.D.

Richard Bebjak

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor:	Richard Bebjak
Název práce:	Paralelní Programování v Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Jan Laštovička, Ph.D.
Počet stran:	20
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Richard Bebjak
Title:	Parallel programming in Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Jan Laštovička, Ph.D.
Page count:	20
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Synopsis

Kľúčové slová:

Keywords:

Děkuji, děkuji, děkuji.

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Čo je to Communication Sequential Process	7
1.1	Základné myšlienky CSP	7
1.2	Komunikácia v CSP	7
1.3	Výrobca a spotrebiteľ CSP	8
2	Goroutine	9
2.1	Model a plánovač goroutine	10
3	Komunikačné kanály	10
3.1	Druhy komunikačných kanálov	11
3.2	Implementácia	11
4	Synchronizačné primitíva	13
4.1	WaitGroup	13
4.2	Mutex a RWMutex	14
5	Select	16
6	Chat server	16
6.1	Klientské spojenie	16
6.2	Klientské požiadavky	16
7	Posielanie správ	17
	Literatúra	20

Zoznam tabuliek

1 Čo je to Communication Sequential Process

Communication Sequential Process (CSP) je základný teoretický prístup ku súbežnému programovaniu.[1] Tento prístup definuje ako môžu súbežne bežiacie procesy spolupracovať a komunikovať. Na rozdiel od tradičných prístupov založených na zdialanej pamäti, CSP je založené na komunikácii medzi procesmi ako hlavným mechanizmom koordinácie. Procesy si neposkytujú priamy prístup k pamäti, čím sa znižuje počet chýb súbehu. Základnou myšlienkou CSP je, že program sa skladá z viacerých sekvenčných procesov vykonávaných súbežne, ktoré medzi sebou komunikujú prostredníctvom vstupno-výstupných operácií.

1.1 Základné myšlienky CSP

Hoare navrhol riešenie na základe troch princípov:

- Strážené príkazy
- Paralelná kompozícia procesov
- Komunikácia pomocou vstupu a výstupu

Koncept strážených príkazov zavedený Dijkstrom umožňuje riadiť vetvenie a nedeterministické správanie. Ide o príkazy, ktoré sa vykonávajú až po splnení určitej podmienky. Paralelná kompozícia opisuje program tak, že program môže pozostávať z viacerých procesov, ktoré sa spúšťajú súčasne, vykonávajú sa nezávisle a končia až keď skončia všetky súbežné procesy. Komunikácia pomocou vstupu a výstupu znamená, že procesy spolu komunikujú výhradne pomocou vstupno-výstupných operácií. Procesy tak nezdieľajú pamäť a ich komunikácia prebieha synchronizovane.

V CSP možno chápať každý proces ako samostatnú výpočtovú jednotku s vlastným stavom, ktorá vykonáva príkazy v definovanom poradí. Procesy nie sú izolované, ich spolupráca je kľúčová na dosiahnutie požadovaného výsledku. Ich spolupráca je realizovaná pomocou synchronnej komunikácie, pri ktorej musí byť odosielateľ aj prijímateľ pripravený na výmenu údajov v rovnakom okamihu. Tento prístup eliminuje problémy spojené s chybami súbehu a nekonzistentným stavom pamäte. Tento prístup ale môže viesť k situácii, kedy procesy čakajú bez možnosti pokračovať.

1.2 Komunikácia v CSP

Hlavnou vlastnosťou CSP je prenos dát medzi procesmi. Hodnota odoslaná jedným procesom je priamo priradená premennej v druhom procese. Procesy musia byť pripravené na komunikáciu v rovnakom okamihu. Prenos hodnoty je atomický a synchronizovaný, teda nemôže prísť k čiastočnému prenosu alebo nekonzistentnému stavu. Zjednodušuje uvažovanie o správnosti programu, pretože

eliminuje potrebu zamykania a konflikty pri prístupe k pamäti. Kladú sa ale väčšie nároky na synchronizáciu procesov.

CSP používa príkazy:

- Priradenie hodnoty
- Vstup
- Výstup
- Prázdny príkaz
- Štruktúrovaný príkaz
 - Alternatívny príkaz
 - Repetitívny príkaz

Priradenie hodnoty premennej funguje ako ho poznáme. Vstup a výstup sú synchronizované tak, že proces posielajúci výstupnú operáciu musí čakať kým proces s adekvátnou vstupnou operáciou je pripravený na komunikáciu a naopak. Štruktúrové príkazy umožňujú vytvárať alternatívy a repetitívy. Alternatívny príkaz umožňuje výber z viacerých možností, podľa splnenia podmienky. Umožňuje synchronizovanie medzi viacerými procesmi naraz. Podľa splnenia podmienky, ak nie je splnená žiadna podmienka, čaká. Ak je splnená jedna podmienka vykoná proces priradený danej podmienke. Ak je splnených viacero podmienok, vykoná náhodnú, výber je nedeterministický. Tento príkaz umožňuje modelovať situácie, kde správanie programu závisí od aktuálnych podmienok alebo interakcie s inými procesmi. Repetitívny príkaz predstavuje cyklus alternatívnych príkazov, kedy sa príkaz vykonáva opakovane, kým je splnená aspoň jedna podmienka. Inak sa cyklus ukončí. Tento príkaz umožňuje modelovať situácie, kde program reaguje na prichádzajúce správy alebo udalosti.

1.3 Výrobca a spotrebiteľ CSP

CSP sa dá v tomto príklade chápať ako monitor s ktorým komunikujú viaceré procesy. Procesy musia byť odlišne pomenované napr. „Výrobca“ a „Spotrebiteľ“, alebo odlišný index napríklad „X(0)“. Každá komunikácia musí mať jednoznačne určený svoj zdroj alebo cieľ. Keď je monitor pripravený komunikovať s ktorýmkoľvek používateľským procesom, zväčša ktorý príde prvý, používa strážený príkaz s rozsahom.

$$[(i : 1..100); X(i)?(value\ parameters) \rightarrow \dots; X(i)!(results);]$$

Premenná „i“ sa používa na posielanie výsledkov späť procesu, ktorý komunikáciu inicioval.

Ako monitor nie je pripravený na komunikáciu od konkrétneho procesu napríklad „X(j)“, môže byť vstupný príkaz doplnený o booleovskú podmienku,

takzvanú stráž. Napríklad dve po sebe idúce požiadavky od rovnakého procesu, môžu byť zablokované pomocou podmienky „j=0“.

$$*[(i : 1..100)i \neq j; X(i)?(parameters) \rightarrow \dots; i := j;]$$

V tomto prípade bude výstup procesu „X(j)“ odložený, kým nebude spracovaná požiadavka iného procesu. Podobne je možné použiť podmienky na oddialenie prijatia vstupov, ktoré by porušili plánovacie obmedzenia. [1]

2 Goroutine

Goroutine je funkcia ktorá sa vykonáva súbežne. Nie sú to vlákna operačného systému ani vlákna riadené runtime prostredím. Ide o vyššiu abstrakciu nazývanú korutiny v jazyku Go goroutine. Goroutines sú súbežné podprogramy, ktoré nemôžu byť prerušené zvonka. Obsahujú však body, v ktorých môžu byť pozastavené a znovu spustené. Na rozdiel od vlákien je prepínanie medzi goroutine rýchlejšie približne o 1,242 μ s.[2] Goroutine sa dajú spustiť pomocou kľúčového slova go.

```
func main{
    go SayHello()
}

func SayHello(){
    fmt.Println("Hello")
}
```

Je možné goroutine implementovať ako anonymné funkcie.

```
go func(){
    fmt.Println("Hello")
}
```

Ako anonymné funkcie musia byť zavolané okamžite. Vieme goroutine priradiť do premennej.

```
sayHello := func(){
    fmt.Println("Hello")
}

go sayHello()
```

Goroutine sú špecifické pre jazyk Go, pretože sú integrované do runtime prostredia Go. Goroutine si sami neurčujú body pozastavenia. Runtime Go sleduje ich správanie a automaticky ich pozastaví, a znovu spustí, keď sú opätovne pripravené pokračovať. Goroutine bežia súbežne, čo nie je ich vlastnosťou, preto potrebujú mechanizmus, ktorý im umožní bežať a vykonávať sa súbežne.

2.1 Model a plánovač goroutine

Go používa plánovač „M:N“ a model súbežnosti fork-join.[2] Teda na „M“ vláknach sa vykonáva „N“ goroutine. Goroutine sú mapované na tieto vlákna a go runtime sa stará o ich rozdelenie a efektívne vykonávanie. Stará sa o ich spúšťanie, ak je nejaká goroutine blokována tak go runtime môže spustiť inú.

Podľa modelu fork-join, „Fork“ znamená, že program môže vytvoriť novú goroutine, ktorá sa vykonáva súbežne s rodičovským kódom. A „Join“ znamená, že sa tieto vetvy spoja v mieste nazývanom join point. 3 Spájanie vetiev nie je funkciou programu, preto je nutná synchronizácia pri goroutine.

3 Komunikačné kanály

Komunikačné kanály predstavujú jeden zo základných mechanizmov synchronizácie a komunikácie medzi goroutines. Sú zabudované referenčným typom, ktorý umožňuje bezpečný prenos dát. Ich návrh vychádza z Communicating Sequential Processes, ktorý zdôrazňuje komunikáciu medzi procesmi miesto zdieľania pamäte.[2]

Komunikačné kanály sa deklarujú pomocou kľúčového slova `chan` a vždy sa deklarujú aj s určitým typom. To znamená, že kanál môže prenášať hodnoty iba tohto typu. Komunikačné kanály patria k referenčným typom, preto ich deklarácia nevytvára potrebnú pamäťovú štruktúru. Pamäť sa alokuje pomocou zabudovanej funkcie `make`.

```
make(chan type, buffer_size)
```

Takto vytvorený komunikačný kanál umožňuje zapisovanie a čítanie hodnôt. Zápis do komunikačného kanála umožňuje operátor `<-`, ktorý sa umiestňuje na pravú stranu názvu komunikačného kanála, pre čítanie sa operátor umiestňuje na ľavú stranu názvu komunikačného kanála.

```
channel <- data  
variable = <- channel
```

Dôležitou vlastnosťou komunikačného kanála je jeho blokujúce správanie. Ak goroutine zapisuje do komunikačného kanála, ktorý je plný, je jej proces blokový kým iná goroutine z komunikačného kanála hodnotu neprečíta. Podobne čítanie z prázdneho komunikačného kanála je blokové kým nie je do komunikačného kanála zapísaná nová hodnota.

Komunikačný kanál je možné zavrieť pomocou funkcie `close`. Pri čítaní z kanála je možné získať druhú návratovú hodnotu typu `bool`, ktorá indikuje jeho otvorenie. Uzavretie teda signalizuje prijímateľovi, že do komunikačného kanála nebude viac zapisované. Ak odosielateľ pošle dáta do uzavretého komunikačného kanálu, program vráti error `panic: send on closed channel`. Po uzavretí komunikačného kanálu, prijímateľ nečaká na hodnotu ale obdrží prázdnu hodnotu a hodnotu „False“ z druhej návratovej hodnoty. Nasledujúci príklad demonštruje použitie.

```

1      var channel = make(chan int , 1)
2
3      go func(){
4          channel <- 42
5      }()
6
7      result := <- channel
8      close(channel)

```

Na riadku jedna si vytvoríme komunikačný kanál s referenčným typom `int` a veľkosťou buffera jedna. Na riadku tri spustíme novú goroutine. Na riadku štyri v goroutine uložíme do komunikačného kanála hodnotu 42. Na riadku sedem hlavné vlákno čaká na dáta z komunikačného kanála. Ak dáta prídu, uloží do premennej „result“ dáta z komunikačného kanála, v tomto prípade 42. Na riadku osem uzavrieme komunikačný kanál.

3.1 Druhy komunikačných kanálov

Každý komunikačný kanál môže byť bufferovaný alebo nebufferovaný. Nebufferovaný komunikačný kanál nemá žiadnu kapacitu a každá operácia zápisu musí byť okamžite spravovaná operáciou čítania.

Bufferovaný komunikačný kanál sa definuje pomocou voliteľného druhého argumentu funkcie `make`. Bufferovaný komunikačný kanál obsahuje buffer s definovanou kapacitou, ktorá umožní uložiť určitý počet hodnôt do buffera bez okamžitého čítania. Bufferovaný komunikačný kanál zablokuje zápis až v momente keď je buffer plný.

Jazyk Go umožňuje definovanie jednosmerných kanálov, ktoré podporujú iba zápis alebo čítanie. Tieto komunikačné kanály sa používajú ako parametre funkcií, aby určili akým spôsobom sa komunikačný kanál v danej časti programu používa. Tento mechanizmus zvyšuje bezpečnosť programu, pretože kompilátor dokáže zachytiť nesprávne použitie.

3.2 Implementácia

V runtime prostredí je komunikačný kanál implementovaný ako štruktúra `hchan`. Obsahuje počet hodnôt v kanáli, maximálnu kapacitu, ukazovateľ na buffer, fronty čakajúcich goroutine a synchronizačný zámok. Zjednodušene štruktúra ob-

	qcount uint	Počet prvkov v bufferi
	dataqsiz uint	Veľkosť jednej hodnoty na bufferi
	buf unsafe.Pointer	Pointer na buffer
	sendx uint	Index pre zápis
sahuje:	recvx uint	Index pre čítanie
	recvq waitq	Čakajúci prijímatelia
	sendq waitq	Čakajúci odosielatelia
	closed uint32	Hodnota určujúca či je komunikačný kanál otvorený alebo zatvorený
	lock mutex	Mutex na prístup k bufferu

Táto štruktúra ukazuje, že komunikačný kanál funguje ako kruhový buffer, doplnený o mechanizmi blokujúce goroutine.[3] Implementácia front „sendq“ a „recvq“ je hlavný prvok na synchronizáciu goroutine. Kedy odosielateľ čaká na príjemcu a príjemca čaká na odosielateľa. To zabezpečuje bezpečnú synchronizáciu a absenciu problému súbehu zámkov v aplikačnom kóde.

Komunikačný kanál má operácie prijímanie a odosielanie hodnoty. Komunikačný kanál odosiela a prijíma dve hodnoty. Posielanú hodnotu a hodnotu typu bool, ktorá značí, či je kanál otvorený. Odosielanie hodnoty je implementované funkciou chansend. Jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene, rozlišuje tri stavy. Ak existuje čakajúci príjemca, teda je hodnota v recvq goroutine, hodnota sa prenesie priamo bez použitia bufferu.

```
if receiver exist:
    copy data directly to receiver
    wake up receiver
```

Najrýchlejší scenár, nakoľko sa neukladá hodnota do bufferu. Ďalší stav, ak neexistuje čakajúci príjemca ale existuje voľné miesto v bufferi. Tak sa hodnota uloží na buffer a zvýši sa počítadlo.

```
if buffer not full:
    store value in buffer
    increment qcount
```

Tretí stav ak neexistuje čakajúci príjemca a neexistuje voľné miesto na bufferi. V takom prípade sa goroutine uloží do fronty „sendq“ a čaká na uvoľnenie miesta na bufferi.

Prijímanie hodnoty je implementované funkciou chanrecv. Rovnako jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene rozlišuje tri stavy. Ak existuje čakajúci odosielateľ, teda je hodnota v sendq, hodnota sa prenesie priamo bez použitia bufferu.

```
if sender exist:
    receive value directly
    wake sender
```

Ak neexistuje čakajúci odosielateľ ale buffer obsahuje aspoň jednu hodnotu, je táto hodnota načítaná a zníži sa hodnota počítadla.

```

if buffer not empty:
    read value from buffer
    decrement qcount

```

Ak neexistuje čakajúci odosielateľ a buffer neobsahuje žiadnu hodnotu. V takom prípade sa goroutine uloží do fronty „recvq“ a čaká na odosielateľa.

Operácia uzavretia komunikačného kanála nastaví „closed“ na hodnotu „False“. Zároveň sa odblokujú všetky čakajúce goroutine v „recvq“. Goroutine v prvej návratovej hodnote obdržia prázdnu hodnotu, čo nie je vždy chcené a je potrebná reakcia programu na druhú návratovú hodnotu. Pri odosielaní na uzavretý komunikačný kanál, program vráti chybu.

4 Synchronizačné primitíva

Jazyk Go poskytuje sync balík, ktorý obsahuje synchronizačné primitíva. Tieto synchronizačné primitíva sú používané pri nízkoúrovňovom prístupe do pamäte. Rozdiel medzi ostatnými jazykmi a jazykom Go je, že Go poskytuje novú sadu synchronizačných primitív pre súbežnosť nad sadou primitív pre synchronizáciu prístupu k pamäti.

4.1 WaitGroup

Je to nástroj z balíka sync, ktorého účelom je umožniť programu počkať na dokončenie skupiny goroutine pred tým, než bude pokračovať. Mechanizmus sa využíva v prípade, že sa vykonáva viacero nezávislých úloh. WaitGroup si môžeme predstaviť ako bezpečný čítač. Tento čítač eviduje počet goroutine, ktoré ešte nedokončili svoju činnosť. Čítač sa zvyšuje pomocou metódy `Add(n)`, kde „n“ udáva počet goroutine, ktoré sa majú vykonať. Čítač sa znižuje pomocou metódy `Done()`. Program je zablokovaný na metóde `Wait()`, kým hodnota čítača nedosiahne nulovú hodnotu. Nasledujúci príklad demonštruje použitie.

```

1      var wg sync.WaitGroup
2
3      for i := 0; i < 10; i++ {
4          wg.Add(1)
5          go func(i int) {
6              defer wg.Done()
7              fmt.Println("Goroutine", i)
8              time.Sleep(200)
9          }(i)
10     }
11     wg.Wait()
12     fmt.Println("End")

```

Na prvom riadku si definujeme WaitGroup. Následne v cykle na riadku tri zvýšime hodnotu WaitGroup o jedna. Na riadku štyri pustíme novú goroutine. V

goroutine na riadku päť zabezpečíme, že po dokončení činnosti, zníži hodnotu `WaitGroup` o jedna. Následne vykoná vypís a čakanie pre simuláciu paralelného výpočtu. Na riadku jedenásť program čaká, kým hodnota `WaitGroup` bude nulová. následne vypíše „End“.

4.2 Mutex a RWMutex

Mutex je nástroj z balíka `sync`, ktorého účelom je chrániť časť programu so zdieľanými zdrojmi, ku ktorým môže pristupovať iba jedna goroutine. Pred vstupom do časti programu so zdieľanými zdrojmi, musí goroutine získať zámok pomocou metódy `Lock()`. Po ukončení činnosti v danej časti programu, uvoľní zámok pomocou metódy `Unlock()`. Počas držania zámku, nemôže iná goroutine vstúpiť do tejto časti programu. `RWMutex` rozširuje `Mutex` o zámok, ktorý rozlišuje operácie čítania a zápisu. Metódy na získanie zámku `RLock()` / `Lock()`, metódy na uvoľnenie zámku `RUnlock()` / `Unlock()`. `RWMutex` umožňuje získať zámok pre čítanie viacerým goroutine, ale pre zápis umožňuje získať zámok jednej goroutine. Počas zápisu nie je umožnené získať zámok pre čítanie. Pri zápise čaká kým sú uvoľnené všetky zámky. Nasledujúci príklad demonštruje použitie.

```
1      var wg sync.WaitGroup
2      var rw sync.RWMutex
3
4      observer := func(i int) {
5          defer wg.Done()
6          rw.RLock()
7          fmt.Println("Reading started", i)
8          time.Sleep(400 * time.Millisecond)
9          fmt.Println("Reading ends", i)
10         rw.RUnlock()
11     }
12
13     writer := func(i int) {
14         defer wg.Done()
15         defer rw.Unlock()
16         rw.Lock()
17         fmt.Println("Writing started", i)
18         time.Sleep(400 * time.Millisecond)
19         fmt.Println("Writing ended", i)
20     }
21
22     for i := 0; i < 100; i++ {
23         wg.Add(1)
24         if i%25 == 0 {
25             go writer(i)
26         } else {
```

```

27                                     go observer(i)
28                                 }
29         }
30     wg.Wait()
31     fmt.Println("End")

```

Na riadku dva si definujeme RWMutex. Na riadku štyri si definujeme funkciu čitateľa, ktorá sa pokúsi získať zámok pre čítanie vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok pre čítanie. Na riadku trinásť si definujeme funkciu zapisovateľa, ktorá sa pokúsi získať zámok, vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok. Na riadku dvadsaťdva je cyklus, v ktorom voláme funkciu zapisovateľa ak „i“ modulo 25 je nula, inak voláme funkciu čitateľa. Následne počkáme na dokončenie všetkých goroutine a vypíšeme, že sme skončili. Výsledok tohto kódu bude približne.

```

" Writting started 0 "
" Writting ended 0 "
" Reading started 1 "
" Reading started 2 "
" Reading started 5 "
...
" Reading ended 98 "
" Reading ended 97 "
" Reading ended 99 "
" Writting started 50 "
" Writting ended 50 "
" Writting started 75 "
" Writting ended 75 "

```

Z tohto výsledku vidíme, že RWMutex uprednostňuje čitateľov. Vynútenie zámku pre zapisovateľa je možné pomocou ďalších synchronizačných nástrojov, ako napríklad pomocou komunikačných kanálov.

5 Select

6 Chat server

Navrhnutý prototyp serveru je založený na architektúre klient-server, kde server zabezpečuje výmenu správ medzi pripojenými klientmi. Komunikácia medzi klientom a serverom je realizovaná cez Transmission Control Protocol (TCP), ktorý zabezpečuje spoľahlivý prenos dát, zachovanie poradia správ a detekciu chýb pri prenose. Server je implementovaný v programovacom jazyku Go, využívajúc jeho vstavanú podporu pre súbežné programovanie prostredníctvom goroutine. Hlavná úloha servera spočíva v prijímaní nových klientskych spojení, spracovaní ich požiadavkov a správe komunikačných miestností.

6.1 Klientské spojenie

```
for {  
    conn, err := ln.Accept()  
    go clientHandle.HandleConnection(conn)  
}
```

Pri spustení server vytvorí TCP listener „ln“. Následne v nekonečnom cykle čaká na nové pripojenie, ak je pripojenie úspešné tak vytvorí pre to spojenie novú goroutine. Týmto je zabezpečené, že každý klient je obslužený súbežne a bez blokovania prijímania ďalších klientov.

Každý klient je obslužený na samostatnej goroutine, ktorá vykonáva funkciu `HandleConnection`. Toto riešenie predstavuje z pohľadu architektúry model:

- jeden hlavný serverový proces
- jedna goroutine na klienta

Riešenie je efektívne, pretože goroutine sú ľahšie ako klasické vlákna a umožňujú vysokú mieru škálovateľnosti.

6.2 Klientské požiadavky

Po pripojení je od klienta vyžiadané meno. Následne v nekonečnom cykle prvá podmienka pripojí klienta do komunikačnej miestnosti a v následných cykloch spracúva požiadavky:

- `/fetch` Získanie nových správ v komunikačnej miestnosti
- `/fetchAll` Získanie správ zo všetkých navštívených komunikačných miestností
- `/broadcast` Poslanie správy do všetkých navštívených komunikačných miestností

- `exit` Opustenie komunikačnej miestnosti
- `logout` Odhlásenie sa zo serveru. Teda ukončenie nekonečného cyklu.
- `default` Poslanie správy

7 Posielanie správ

Problémom pri posielaní správ je, aby sa jednotlivé správy uložili do databázy správne, a aby nevznikol problém súbehu. Problém sme riešili pomocou komunikácie medzi procesmi s použitím komunikačného kanála. Využili sme jeho blokujúce správanie a prenos dát medzi procesmi. Na správu správ sme použili štruktúru `Message`, kde sa eviduje odosielateľ a obsah správy. Pre každú komunikačnú miestnosť sme použili štruktúru `ChatLog`, ktorá obsahuje `Message` v zozname ako poslané správy v danej komunikačnej miestnosti. Táto štruktúra obsahuje aj komunikačný kanál, pomocou ktorého sa správy ukladajú súbežne.

```

1         for msg:= range cl.writeCh {
2             cl.Message = append( cl.Message , msg)
3         }

```

Týmto je zabezpečené, že komunikačný kanál čaká kým doň nie je zapisované. Ak chce klient zapísať správu, tak komunikačným kanálom pošle správu, ktorá sa pridá do zoznamu.

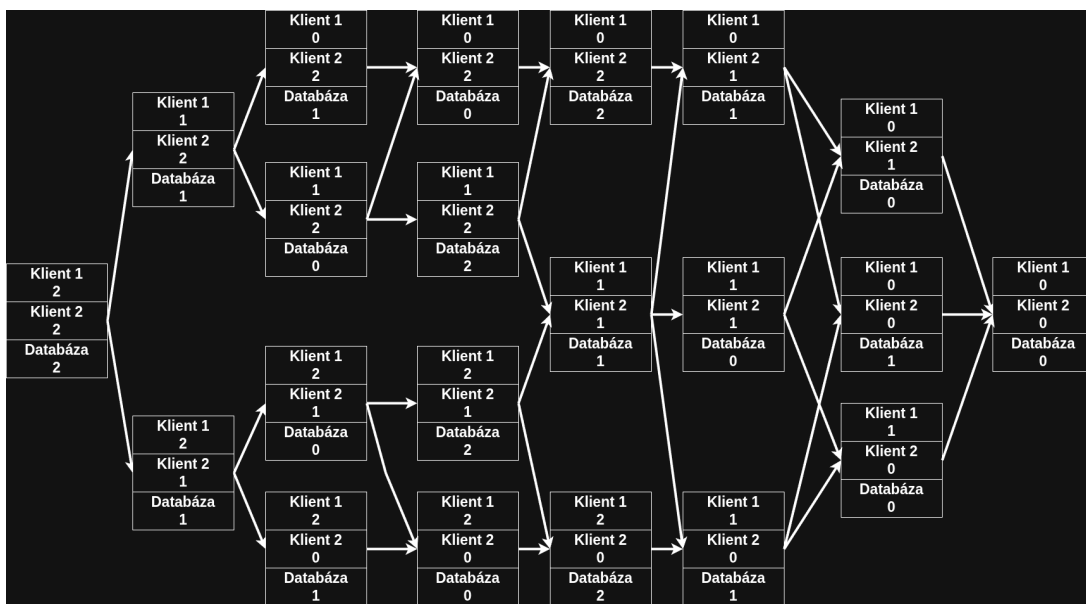
```
cl.writeCh <- message.Message{User: user , Message: text}
```

Klient posiela správu cez kanál. Aj keď môže viacero klientov poslať správu súčasne, komunikačný kanál umožňuje poslanie iba jednej správy, ktorá sa spracuje predtým ako je umožnené ďalšiemu klientovi poslanie správy cez komunikačný kanál. Je preto klient pozastavený a čaká kým je opäť program pripravený spracovať ďalšiu správu. Zapísané v pseudokóde:

go handler() go channel()	
HANDLER	CHANNEL
ch <- message return	for { message := <- ch save(message) }

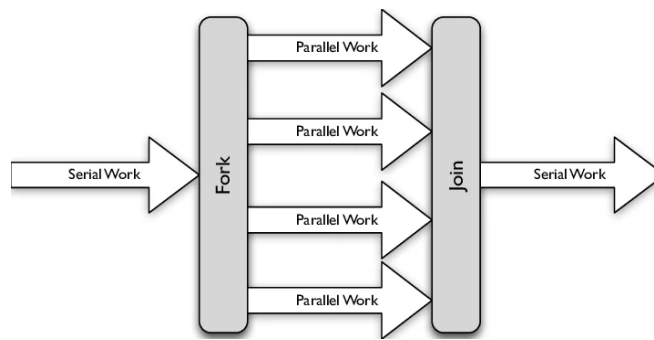
Obr. 1: Pseudokód posielania správ

Vytvoríme groutine pre ukladanie dát, ktorá ukladá správy do databázy. Ďalej vytvoríme goroutine predstavujúcu obsluhu klienta. Vidíme, že aj pri viacerých klientoch nevznikne problém súbehu a dáta sa zapíšu do databázy správne. Využívame blokovacie správanie komunikačného kanála, kedy pri prijatí viacerých správ odosielateľ čaká, kým je prijímateľ pripravený spracovať dáta. Dokážeme si to na príklade dvoch procesov klientov a procesu databázy.



Obr. 2: Dôkaz posielania správ

Tu vidíme, že procesy klientov sú blokované, kým proces ukladania dát je pripravený prijať dáta. Preto vždy dôjdeme k požadovanému výsledku.



Obr. 3: Model fork-join

Literatúra

- [1] Hoare, C. A. R. Communicating Sequential Processes. In Graham, S. L.; Rivest, R. L. (ed.). *Programming Techniques*. New York: Academic Press, 1978.
- [2] Cox-Buday, Katherine. *Concurrency in Go*. First. Sebastopol, CA: O'Reilly Media, 2017. 224 str. ISBN 978-1-491-94119-5.
- [3] Go Authors. *Go Runtime Source Code: runtime/chan.go*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02.
- [4] Beckingsale, David. *Towards Scalable Adaptive Mesh Refinement on Future Parallel Architectures*. 2015.